A Project-Based Approach to Teaching Game Engine Architecture Course

Prashant Goswami[†] and Nan Huang

Blekinge Institute of Technology, Sweden

Abstract

Game engine architecture (GEA) underpins the design of complex, performance-critical interactive systems in modern game development. This paper presents the design and evolution of a fourth-year elective course on Game Engine Architecture *taught within a five-year integrated Master's engineering program in Game Technology at Blekinge Institute of Technology, Sweden. The course combines lectures, programming assignments, and a final project to support architectural analysis, subsystem implementation, and system-level integration. The paper outlines how the course has been iteratively refined across multiple offerings, including changes to assignments, lecture content, and assessment. It summarizes student evaluation data and discusses instructor reflections on learning patterns, challenges, and course adaptations, providing a grounded account of teaching GEA as an advanced software architecture subject.*

CCS Concepts

• ***Software and its engineering*** → ***Software architecture;*** *Software design engineering; Software creation and management;* • ***Social and professional topics*** → *Computer science education;*

1. Introduction

Modern game development relies on increasingly complex software systems that must balance visual effects, performance, scalability, maintainability, and rapid iteration. The global game market has grown into a major creative and technological industry, with annual revenues exceeding USD 180 billion worldwide (2023), surpassing the movie and music industries combined (New23). At the core of modern game production lies the game engine (LJ02), whose architecture fundamentally shapes both the development process and the resulting interactive experience. As game engines have evolved from tightly coupled, domain-specific systems into modular, data-driven platforms supporting diverse genres and hardware targets, the need for deeper architectural understanding has become central to professional game development education.

In response to this evolution, universities offering advanced game engineering programs face the challenge of teaching game engine architecture (GEA) not merely as a collection of techniques, but as an evolving design space (Tho11) involving trade-offs, constraints, and long-term consequences. This challenge is particularly pronounced at the master's level, where students are expected to move beyond implementation and develop the ability to analyze, evaluate, and justify architectural decisions in complex software systems.

This paper presents the design and longitudinal evolution of a fourth-year elective course on *GEA* taught within the five-year integrated Master's engineering program in Game Technology (Civilingenjör i Spel- och programvaruteknik) at Blekinge Institute of Technology (BTH), Sweden. The course targets senior undergraduate students with prior experience in three-dimensional (3D) graphics, game development, and software architecture, and is intended to support the transition from implementation-focused development to architectural reasoning in the context of game engines.

The contribution of this paper is a structured account of how such a course has been iteratively developed and refined over multiple offerings. It describes the rationale behind the course structure, including the introduction of analysis-driven assignments, subsystem-focused implementation tasks, and a final integration project, as well as updates to lecture content reflecting changes in the field. The paper further reports enrollment trends and student evaluation data across several iterations and discusses instructor reflections on observed learning patterns, challenges, and course adaptations. Together, these elements provide a grounded perspective on teaching game engine architecture as an advanced software architecture subject and highlight considerations for maintaining relevance in a rapidly evolving domain.

[†] Chairman Eurographics Publications Board

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.
This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

DOI: 10.2312/eged.20261000

delivered by EUROGRAPHICS DIGITAL LIBRARY www.eg.org diglib.eg.org

Table 1: *Representative university courses focusing on game engine internals and engine architecture.*

Country	University	Course Code	Course Name	Credits	Level	Duration
Sweden	University of Gothenburg (Uni26b)	DIT572	Game Engine Architecture	7.5	Master's	Quarter
Sweden	Luleå University of Technology (Lul26)	W0047E	Game Engine Architecture	15	Bachelor's	Quarter
Finland	University of Helsinki (Uni26c)	CSM14206	Game Engine Architecture	5	Master's	Quarter
USA	University of Michigan (Uni26d)	EECS 498	Game Engine Architecture	4	Upper-Level	Semester
USA	University of Nevada, Reno (Uni26e)	CS381	Game Engine Architecture	3	Bachelor's	Semester
Sweden	Blekinge Institute of Technology (BTH)	DV2591	Game Engine Architecture	7.5	Master's	Quarter

2. Background

Globally, game development education has expanded rapidly over the past two decades, driven by the growth of the interactive entertainment industry and the increasing adoption of game technologies in adjacent domains such as simulation, virtual production, and serious games (SW20; New23; D*18). International curricula often emphasize practical engine usage, commonly through commercial engines such as Unity or Unreal Engine, and gameplay programming, while explicit instruction in engine architecture remains comparatively underrepresented or embedded implicitly within project-based courses (D*19).

In Sweden, this challenge is framed within a national higher education policy that emphasizes:

- research-informed yet real-life applicable education,
- progression from practical skills to analytical competence,
- alignment with societal and industrial needs, and
- student-centered and project-based learning.

According to the Swedish Higher Education Act (Högskolelagen) and Higher Education Ordinance (Högskoleförordningen), advanced-level engineering education must enable students to analyze and evaluate complex phenomena, integrate knowledge across domains, make informed judgments under constraints, and critically reflect on technological solutions and their broader implications (Swe92; Swe93).

Within this policy framework, teaching game engine architecture is not solely about technical proficiency, but about fostering engineering judgement, architectural reasoning, and critical reflection, competencies that are explicitly required at the advanced level (A1N) and are increasingly valued by both the Swedish and international games industry.

3. Related work

Teaching game engine architecture sits at the intersection of two established literatures: (1) game development and game engine courses that address engine internals and engine construction, and (2) software architecture education, which studies how architectural reasoning, design trade-offs, and communication in large-scale systems can be effectively taught. The following paragraphs summarize what is relevant from both areas and identify gaps in existing published accounts that this paper aims to address.

3.1. Courses on game engines

Several universities internationally offer courses that explicitly focus on game engines or closely related topics, combining lectures on core engine subsystems with hands-on development tasks. Tab. 1 summarizes representative examples from Europe and North America, illustrating how such courses are positioned across degree levels, credit loads, and academic calendars.

In the European context, courses at the University of Gothenburg and the University of Helsinki are offered at the Master's level, typically as short, intensive quarter-based modules (5–7.5 ECTS), while Luleå University of Technology offers a substantially larger 15 ECTS Bachelor's level course, indicating a deeper time investment but earlier placement in the curriculum. In contrast, U.S. offerings such as the University of Michigan's EECS 498 and the University of Nevada, Reno's CS381 are semester-long courses, awarded 3–4 credits, and are positioned at the upper undergraduate or advanced undergraduate level.

Across these courses, the pedagogical focus is largely consistent. Course descriptions and syllabi emphasize the implementation and integration of core engine subsystems, such as rendering pipelines, scene and entity management, resource handling, physics integration, and scripting support, often through engine-from-scratch development or subsystem-level projects. Whereas game engine architecture courses are usually implementation-oriented,

From the perspective of software architecture education, these courses provide an important practical context; however, published accounts rarely report how architectural reasoning, trade-off analysis, or design justification are explicitly taught or assessed within game engine curricula.

3.2. Research gap

While such courses are well established across institutions, published accounts reporting empirical observations, student outcomes, and longitudinal course evolution remain limited. This paper, therefore, reports findings derived from the design, delivery, and evaluation of a game engine architecture course, rather than presenting a comparative analysis. The contribution of this work lies in documenting instructional design choices and observed outcomes in context, rather than proposing a new curriculum model or benchmarking approaches across institutions.

4. GEA in the Game curriculum

This section describes how the course is positioned with respect to the program structure, student background, and instructional context. The focus is on providing sufficient context to understand the intended learning level, cohort characteristics, and pedagogical constraints under which the course is delivered.

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

4.1. Program context and student profile

The GEA course is situated within the *Civilingenjör* program in Game Technology at BTH, a five-year integrated Master of Science in Engineering program (300 ECTS). The course is offered as a 7.5 ECTS credit course in the fourth year of study, at an advanced level (second cycle studies or A1N), and targets students who have already acquired substantial experience in both game development and software engineering.

Students who enroll in the course typically have prior exposure to object-oriented and component-based design, game programming, and software architecture concepts through earlier coursework and project-based modules. As a result, the cohort is expected to engage with architectural questions beyond basic implementation concerns, including trade-offs related to performance, extensibility, and maintainability.

Class sizes have varied across course iterations. During the initial offerings of the course (approximately 2015–2018), enrollment typically consisted of around 20 or more students. In later years, cohort sizes decreased, with enrollments ranging from approximately 5 to 12 students per iteration, due to a combination of factors such as program restructuring, the impact of the COVID-19 pandemic, and broader changes in student enrollment patterns. The course is taught to a single cohort of senior students and is delivered primarily in English to accommodate an international student body.

4.2. Learning objectives and prerequisites

According to the official course plan for GEA (Ble19), the intended learning outcomes are structured across the following dimensions:

Knowledge and understanding After completing the course, students are expected to analyze and document different game engine architectures, understand architectural strengths and weaknesses in relation to requirements and constraints, motivate architecture recommendations, and recognize limitations in their own knowledge within the domain of engine architecture.

Skills and abilities Students are expected to implement and extend architectural prototypes, make informed design decisions based on given requirements, conduct relevant performance and architectural measurements, and document and present engine designs and architectural changes in a structured manner.

Values and attitudes Upon completion of the course, students should be able to independently and collaboratively discuss and evaluate alternative engine architectures and critically assess architectural solutions with respect to extensibility and reusability.

Formal prerequisites include prior coursework in software architecture and quality, as well as experience from a larger game development project, ensuring that students enter the course with sufficient background to engage with architectural-level reasoning.

4.3. Course structure and pedagogical approach

The GEA course is designed to combine conceptual instruction with hands-on development through a structured sequence of lectures, assignments, and a final project. The pedagogical approach emphasizes progressive learning, where theoretical concepts are introduced and discussed in lectures and subsequently applied, analyzed, and evaluated through practical work. Together, these components are intended to support architectural reasoning, design justification, and reflective evaluation in the context of complex game engine systems.

Table 2: *Overview of lecture themes in the GEA course.*

Lecture	Main Topics
L1	Course overview, games and game engines, industry structure, game genres, runtime engine architecture, historical evolution
L2	Tools and asset pipelines, asset conditioning, tool architecture, engine support systems, memory management, caching, localization
L3	File systems, resource management, resource lifetimes, time measurement, hardware architectures
L4	Human interface devices, in-game debugging, development tools, client–server and peer-to-peer architectures, CUDA & GPU basics
L5	Gameplay systems, game world structure, progression vs. emergence, dynamic game elements, Physical simulation
L6	Runtime object models, dynamic objects, object- vs. property-centric architectures, asynchronous loading, world chunking
L7	Event systems and messaging, rendering pipeline, camera and lighting models, shadows and visual effects

4.3.1. Lectures

The lecture component of the GEA course provides the conceptual framework, also aligned with the reference literature book by Jason Gregory (Gre18), that supports the assignments and the final project. Lectures cover a broad range of topics spanning engine architecture, runtime systems, tools and pipelines, and gameplay and rendering concepts, with an emphasis on architectural structure and design trade-offs. Real-time physical simulation for games and related GPU programming concepts are addressed across multiple lectures. These include discussions of frameworks such as NVIDIA PhysX (NVI26), as well as examples from the author's research on physics-based GPU methods for real-time computer graphics, including work on snow simulation (GNN22; AÖG24), volumetric clouds (GN17), and fluid simulation (GEF15). These topics are introduced in the context of performance-critical gameplay and simulation requirements, which are central to games that rely on realistic physical behavior (HvK19). Tab. 2 summarizes the main themes addressed across the lecture series.

4.3.2. Assignments and project

The GEA course follows a project-based learning model and is structured around three programming assignments and a final project, as defined in the course plan. Students work in fixed groups of three to four throughout the course (which they mostly retain throughout the course), allowing architectural decisions, design

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

trade-offs, and implementation strategies to be discussed and refined collaboratively over time. Tab. 3 summarizes the assignment structure of the GEA course, while Fig. 1 outlines the timeline, outlining the sequence of assignments, the final project, and their associated assessment forms as defined in the course plan.

1. **Assignment 1** focuses on architectural analysis rather than implementation. Student groups select an existing game engine and study its overall architecture, identifying core subsystems and their interactions. In addition, each student performs an in-depth analysis of a selected module and examines its dependencies on other components. To contextualize architectural choices, students also analyze games developed using the selected engine, allowing them to reflect on how architectural decisions support different game genres. This assignment is assessed through an oral presentation and emphasizes architectural understanding and communication.
2. **Assignment 2** addresses low-level memory management. Students implement custom memory allocators, including a pool allocator and a stack allocator, and compare their performance against naive operating system–based allocation strategies. The students are required to choose from some other features, also based on the group size, such as a stomp debugger or a buddy allocator, to be implemented in the memory manager. The goal is to expose students to the performance overheads associated with frequent system-level memory allocation and to motivate why game engines typically manage large memory blocks internally. This assignment combines programming tasks with a written report analyzing performance characteristics.
3. **Assignment 3** focuses on resource handling and asset management. Students design and implement a multi-threaded resource handler capable of creating and unpacking asset bundles, supporting multiple texture and mesh formats. Additional requirements include asynchronous loading and systematic stress testing to evaluate the impact of resource loading and unloading on runtime behavior. Similar to the second assignment, students are presented with a list of choices, such as buffered/unbuffered I/O, asynchronous resource fetching, supporting additional mesh or texture formats, and supporting audio. As with the second assignment, both code and a written report are submitted.
4. **Final project** integrates concepts from the previous assignments by combining the custom memory manager and resource handler into a cohesive prototype. Students analyze memory consumption, performance, and behavior under stress, visualize the results on the running screen, and document their architectural decisions and evaluation results. The project assessment includes a written report and an oral presentation, reinforcing the importance of both technical implementation and architectural reasoning.

Fig. 2 illustrates recent representative outcomes from student projects developed as part of the GEA course. The examples highlight how students instrumented their engines with visualization and diagnostic tools to support architectural analysis rather than focusing solely on rendering results. In particular, the tooling shown in the figure supports the integration of the custom memory manager and resource handler developed in Assignments 2 and 3, respectively, which are combined and evaluated as part of the final project.

Table 3: *Course components and assessment structure (DV2591).*

Component	Focus	Work Type	Assessment	ECTS
Assignment 1	Architecture analysis	Presentation	P/F	1.5
Assignment 2	Memory management	Code + report	P/F	1.5
Assignment 3	Resource handling	Code + report	P/F	1.5
Presentation	Communication	Oral	P/F	1.0
Project	Integrated prototype	Code + report + oral	A–F	2.0

The left image shows a memory management visualization tool that exposes allocator behavior at runtime, including stack, pool, and buddy allocators, live allocations, and memory usage patterns. Such tooling enables students to reason about allocation strategies, fragmentation, and lifetime management under varying workloads. The right image presents an integrated engine prototype used for resource management and stress testing, combining asset loading, runtime diagnostics, and interactive control to evaluate system behavior under load. Together, these examples demonstrate how students use visualization and measurement to support architectural reasoning, evaluation, and reflection during development.

4.4. Rationale for course structure

The assignments and final project in the GEA course are designed to scaffold architectural reasoning, progressing from analysis and conceptual understanding toward implementation, integration, and evaluation. Each component targets a specific architectural concern while contributing to a coherent learning trajectory.

The initial assignment establishes a shared architectural vocabulary by focusing on the analysis of existing engines and their application contexts. Subsequent assignments address memory management and resource handling, two subsystems that are central to efficiency, flexibility, and control in modern game engines. Implementing these components allows students to contrast operating system–level services with engine-level solutions and to reason about performance, resource lifecycles, and runtime behavior through measurement and stress testing.

The final project integrates the developed subsystems into a cohesive prototype, requiring students to evaluate system behavior under realistic constraints. By assigning a graded assessment to this integration task, the course emphasizes architectural reasoning, documentation, and evaluation alongside functional correctness.

In terms of staffing, the course was initially delivered with the course responsible serving as the main lecturer, supported by an additional instructor who conducted separate weekly sessions focused on progress discussions and assignment support. Over time, these dedicated help sessions were integrated into the lecture schedule, with time allocated at the end of lectures for questions and guidance, all handled solely by a single instructor. This change provided students with immediate support after the introduction of core concepts and allowed more flexible time for independent work on assignments.

4.5. Grading criteria

The grading structure of the GEA course follows the assessment model defined in the official course plan and distinguishes between formative components and a summative final project. The course

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

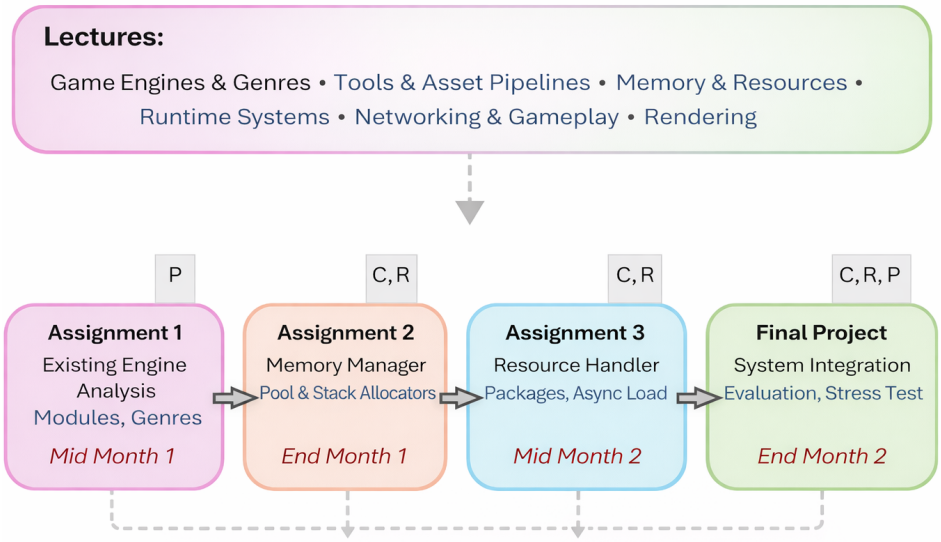


Figure 1: *Progression of assignments and project milestones in the GEA course. The structure scaffolds learning from architectural analysis through subsystem implementation and integration, culminating in evaluation and presentation. Each assignment also carries one or more examination modes (deliverables) as P: Presentation, R: Report, and/or C: Code.*

includes three assignments and a presentation that are assessed on a pass/fail basis, alongside a final project that is graded on a scale from A to F and determines the overall course grade.

The pass/fail assignments are designed to ensure that all students engage with core architectural concepts and acquire essential implementation experience. Assignment 1, assessed through an oral presentation, focuses on architectural analysis and communication. Assignments 2 and 3 require both programming submissions and written reports and are evaluated based on correctness, completeness, and the ability to analyze architectural implications.

The final project serves as the primary summative assessment. It is evaluated using the full A–F grading scale and requires students to integrate previously developed subsystems into a cohesive prototype, document architectural decisions, and analyze system behavior under realistic and stressed conditions. The project assessment emphasizes architectural reasoning, justification of design choices, and clarity of documentation in addition to functional correctness. The final course grade is awarded only after all assessment components have been successfully completed.

4.6. Evolving the course

Across multiple iterations, the GEA course has been incrementally updated in response to changes in program structure, the evolving landscape of game development, student backgrounds, and accumulated instructional experience. Rather than undertaking major redesigns, the course has evolved through targeted adjustments intended to strengthen architectural reasoning, balance workload, and improve alignment between lectures, assignments, and assessment.

Lecture content has been continuously revised to incorporate recent developments and emerging topics that are not yet reflected in commonly used course textbooks, ensuring that the material remains aligned with current practices in game engine development. Apart from this, in the following, we describe two concrete examples of such updates that illustrate how the course structure and content were adapted over time.

4.6.1. Adding the first assignment

In early iterations of the course, the emphasis was placed on subsystem implementation and the final integration project, with no dedicated assignment focused on architectural analysis. While this supported hands-on development, it provided limited opportunity for students to establish a shared architectural vocabulary before engaging in implementation tasks. To address this gap, an initial assignment (Assignment 1) centered on architectural analysis of an existing game engine was introduced.

In this assignment, students select and analyze a widely used game engine, such as Godot (God26), Unity (Uni26a), Unreal Engine (Epi26), or CryEngine (Cry26). By analyzing how these engines support different game genres, students are encouraged to reason about architecture at a system level and to articulate architectural concepts using appropriate terminology. This addition aims to establish a common analytical foundation early in the course and better prepare students for subsequent assignments.

4.6.2. Adding a Virtual Reality (VR) lecture

The increasing use of virtual, augmented, and mixed reality in games introduces additional architectural challenges related to interaction, performance, and hardware diversity, which have been discussed extensively in prior work on extended reality (XR) game

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

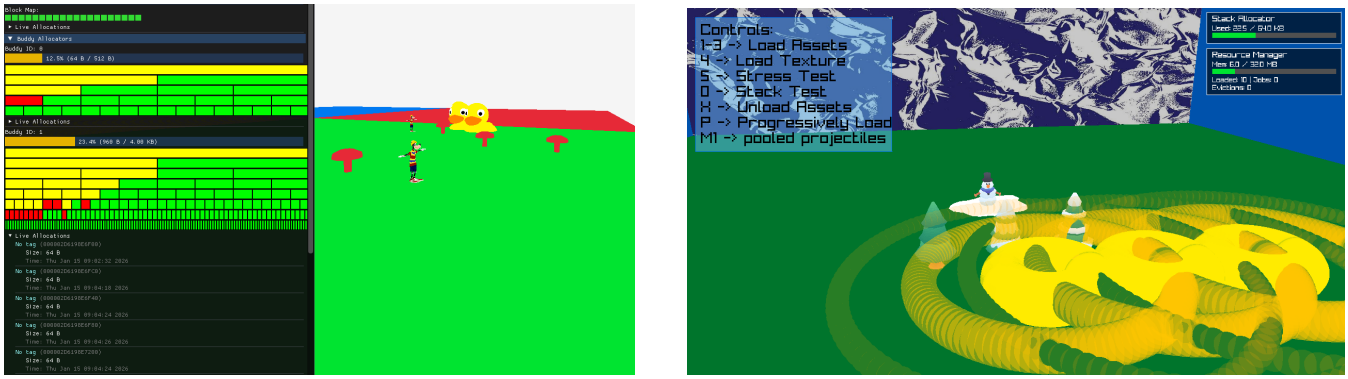


Figure 2: *Student-developed tools and runtime visualization from the project of the GEA course using a game-like environment.* ***Left:*** *A memory management visualization tool showing stack, pool, and buddy allocators with live allocation tracking.* ***Right:*** *An integrated engine prototype demonstrating resource management, stress testing, and runtime diagnostics during execution.*

design and development (Jer16; SPZ18; MH20). Due to the growing importance and the shift of many popular games towards VR, an eighth lecture was added to the curriculum starting 2025.

This lecture was delivered with the help of a teaching assistant (Ph.D. student) already working on immersive XR environments for his research work. A lecture-based introduction to VR and its role in game design aims to establish students' conceptual and technical understanding of how VR characters can benefit game experience, such as immersion, interaction, and presence, which are widely recognized as core components of effective experiences (Ste92; SW97). This framing emphasized the fundamental differences between VR and traditional screen-based games.

The lecture covered essential VR hardware components, including head-mounted displays (HMDs), input devices, and tracking systems, including different tracking approaches. Core VR software components, including real-time rendering, tracking, physics simulation, and interaction systems, were also discussed. This lecture further addressed VR-specific game design challenges, including interaction design, locomotion, 3D user interface design, and user comfort, which have been identified as critical factors affecting usability and simulator sickness in VR environments (Jer15; Lav00).

After completing the theoretical instruction, students participated in a hands-on VR session using the VR game Half-Life: Alyx. Each student was given the opportunity to experience VR gameplay mechanics, such as object manipulation and physics-based interaction. This hands-on approach aligns with experiential learning theory (ELT) (Kol83), which emphasizes learning through concrete experience and reflection.

5. Course evaluation and reflections

This section presents an overview of course evaluation data and instructor reflections collected across multiple iterations of the GEA course. Tab.4 questionnaire consisted of eight items covering key dimensions of the learning process, including course instruction, students' self-evaluation of learning, perceived learning outcomes, and the psychological learning environment. Each item was rated on a 5-point Likert scale, ranging from 0 ("totally disagree") to 4 ("totally agree"). The reported evaluation scores represent the average of students' responses across all items, providing an overall measure of perceived learning experience and course effectiveness.

Tab. 5 summarizes students' overall evaluation scores, enrollment numbers, and response counts across different offerings of the course. The discussion draws on these quantitative indicators, along with qualitative feedback, to highlight both strengths and challenges observed during course delivery. Together, the evaluation results and reflections provide context for understanding how the course structure, learning activities, and assessment approach supported advanced-level architectural learning.

Across all iterations, students were not required to use a specific game engine, allowing them to select or develop engines appropriate to their projects, while C++ (with shaders/CUDA) was used as the mandatory implementation language to ensure a common technical baseline. Over time, the course also underwent structural changes, including a transition from a one-study-period (1 LP) format to a two-study-period (2 LP) format, affecting pacing, workload distribution, and the balance between lectures and project work. These changes are reflected in both enrollment patterns and student feedback and are considered in the reflections that follow.

5.1. Positive reflections

Based on the course evaluation results summarized in Tab. 5, student feedback across multiple iterations indicates a generally positive reception of the course structure and learning objectives. Students frequently highlighted the progressive nature of the assignments as supportive of their understanding of game engine architecture. The introduction of the first assignment helped students develop an understanding of how existing game engines are structured and how their subsystems interact. In the second assignment, students reported a clearer appreciation of the role and importance of custom memory management in game engines, while the third assignment provided hands-on experience with the design and implementation of a resource manager.

Several students noted that the integration of these components

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

Table 4: *Questionnaire for student evaluation, each item is rated on a 5-point Likert scale from 0 (totally disagree) to 4 (totally agree).*

Item	Statement
1	The learning objectives and expected outcomes of the course were clearly communicated.
2	The course content was explained clearly and was easy to understand.
3	This course provided opportunities for me to improve my skills.
4	I actively participated in the course activities and maintained continuous engagement.
5	I received feedback during the course that was helpful for my learning.
6	The instructor created a learning environment (on-campus or online) that encouraged questions and discussion.
7	The learning activities in the course helped me achieve the learning objectives.
8	I felt included and respected during the course.

in the final project made the overall structure and operation of a simplified game engine more apparent. Qualitative feedback further emphasized appreciation for the alignment between lectures and practical work, as well as the opportunity to analyze performance, memory usage, and resource behavior in a realistic engine context. Despite relatively small cohort sizes, the consistently high evaluation scores and response rates in some iterations suggest sustained student engagement and provide useful input for ongoing course refinement.

Students also responded positively to the addition of a dedicated lecture on VR, which was introduced in 2025 to reflect the evolving role of XR technologies in contemporary game development. This lecture helped situate GEA concepts within emerging interaction paradigms and hardware constraints, highlighting how architectural decisions extend beyond traditional screen-based games to immersive and interactive XR experiences.

5.2. Challenges and limitations

At the same time, the evaluation data also reflects several challenges inherent to the course context. Small cohort sizes and varying response rates limit the extent to which quantitative evaluation scores can be generalized across iterations. In addition, qualitative comments point to the demanding workload and steep learning curve associated with low-level engine subsystems such as memory management and resource handling, particularly for students with uneven prior experience. Some students reported difficulties balancing conceptual understanding with implementation effort within the compressed quarter-length format. These observations underscore the trade-offs involved in teaching complex architectural topics in an advanced elective course and highlight the importance of continuous adjustment of scope, pacing, and instructional support.

Some students also reported challenges related to workload distribution in group work, noting that contributions within groups of three to five students were not always evenly balanced. While group-based assignments supported collaboration and discussion, this occasionally led to perceptions of unequal effort among group members.

6. Conclusions

This paper has presented a descriptive account of the design, evolution, and delivery of a fourth-year elective course on Game Engine Architecture within a Master's engineering program in Game Technology. By outlining the course structure, lecture themes, assignments, and assessment approach, and by summarizing student evaluation data and instructor reflections across multiple iterations, the paper provides insight into how GEA can be integrated into an advanced software architecture curriculum.

Table 5: *Overview of course offerings, credits & duration, student enrollment & student evaluation counts, and the course evaluation.*

Year	Duration	Enrolled	Responded	Eval. Score
2023	7.5 ECTS (Quarter)	8	4	3.9 / 4.0
2021	7.5 ECTS (Quarter)	6	4	3.3 / 4.0
2019	7.5 ECTS (Quarter)	5	4	3.8 / 4.0
2016	7.5 ECTS (Semester)	36	28	3.54 / 4.0
2015	7.5 ECTS (Semester)	23	8	3.7 / 4.0

A key observation is that teaching game engine architecture requires continuous adaptation. Technologies, development practices, and target platforms evolve rapidly, while textbooks and formal teaching materials lag behind. As a result, course content must be actively curated and contextualized by the instructor, drawing on current industry practices and emerging developments. This highlights the importance of sustained engagement with the field when designing and maintaining the course.

Looking ahead, further evolution will be necessary to reflect emerging trends in game development. In particular, the growing role of artificial intelligence (AI) in games—spanning gameplay systems, content generation, player modeling, and tool support—suggests a need to integrate AI-related architectural considerations into future iterations of the course to maintain its relevance.

Acknowledgements

The authors would like to acknowledge the significant contributions of Francisco Lopez Luro, who played an important role in the development and teaching of the Game Engine Architecture course during its early years. Francisco was deeply involved in shaping the skeletal course structure and delivery. He is sadly no longer with us, and this work gratefully recognizes his lasting contribution.

The author also thanks the students whose project work is illustrated in Fig. 2: Alexander Cao, Arvid Wahlgren, and Emil Halvarsson (left), and Oliver Toth, Pontus Lindvall, Axel Bergman, and Hugo Waltersson (right), for allowing their project results to be included in this paper. The author also thanks the anonymous

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

reviewers for their constructive comments and suggestions, which helped improve the quality and clarity of this paper.

References

[AÖG24] ANDREASSON S., ÖSTERGAARD L., GOSWAMI P.: Parallel spatiotemporally adaptive DEM-based snow simulation. *Proceedings of the ACM on Computer Graphics and Interactive Techniques 7*, 3 (2024), 1–20. 3

[Ble19] BLEKINGE TEKNISKA HÖGSKOLA: Kursplan: Spelmotorarkitekturer (game engine architecture), dv2591. Internal course syllabus, 2019. Advanced-level course plan, 7.5 ECTS. 3

[Cry26] CRYTEK: CRYENGINE. https://www.cryengine.com, 2026. Accessed 2026. 5

[D*18] DAVIS J. P., ET AL.: Serious games: Applications for training and simulation. *IEEE Computer Graphics and Applications 38*, 2 (2018), 8–11. 2

[D*19] DORMANS J., ET AL.: Teaching game development with commercial game engines. In *Proceedings of the Foundations of Digital Games Conference* (2019). 2

[Epi26] EPIC GAMES: Unreal Engine. https://www.unrealengine.com, 2026. Accessed 2026. 5

[GEF15] GOSWAMI P., ELIASSON A., FRANZÉN P.: Implicit Incompressible SPH on the GPU. In *VRIPHYS 2015-11th Workshop on Virtual Reality Interaction and Physical Simulation* (2015), pp. 23–29. 3

[GN17] GOSWAMI P., NEYRET F.: Real-time landscape-size convective clouds simulation and rendering. In *VRIPHYS 2017-13th Workshop on Virtual Reality Interaction and Physical Simulation* (2017). 3

[GNN22] GOSWAMI P., NORDIN A., NYLÉN S.: Iterative discrete element solver for efficient snow simulation. In *Eurographics Symposium on Parallel Graphics and Visualization (EGPGV), Rome, June 13* (2022). 3

[God26] GODOT ENGINE COMMUNITY: Godot Engine Documentation. https://docs.godotengine.org, 2026. Accessed 2026. 5

[Gre18] GREGORY J.: *Game engine architecture*. AK Peters/CRC Press, 2018. 3

[HvK19] HUMMEL M., VAN KOOTEN K.: Leveraging nvidia omniverse for in situ visualization. In *International Conference on High Performance Computing* (2019), Springer, pp. 634–642. 3

[Jer15] JERALD J.: The vr book: Human-centered design for virtual reality. *The VR Book: Human-Centered Design for Virtual Reality* (2015). URL: https://api.semanticscholar.org/CorpusID:108749274. 6

[Jer16] JERALD J.: *The VR Book: Human-Centered Design for Virtual Reality*. ACM Books, 2016. 6

[Kol83] KOLB D. A.: Experiential learning: Experience as the source of learning and development. URL: https://api.semanticscholar.org/CorpusID:146755699. 6

[Lav00] LAVIOLA J. J.: A discussion of cybersickness in virtual environments. *ACM SIGCHI Bull. 32* (2000), 47–56. URL: https://api.semanticscholar.org/CorpusID:13006130. 6

[LJ02] LEWIS M., JACOBSON J.: Game engines. *Communications of the ACM 45*, 1 (2002), 27. 1

[Lul26] LULEÅ UNIVERSITY OF TECHNOLOGY: Game engine architecture. https://www.ltu.se, 2026. Course syllabus, accessed 2026. 2

[MH20] MOTTELSON A., HORNBÆK K.: Vr games and interaction techniques: Challenges and opportunities. In *Proceedings of the ACM Symposium on Virtual Reality Software and Technology* (2020). 6

[New23] NEWZOO: Global games market report. https://newzoo.com, 2023. Accessed 2026. 1, 2

[NVI26] NVIDIA CORPORATION: NVIDIA PhysX SDK. https://developer.nvidia.com/physx-sdk, 2026. Accessed 2026. 3

[SPZ18] STEED A., PAN Y., ZISCH F.: Lessons learned from designing and developing vr games. In *Proceedings of the IEEE Conference on Virtual Reality and 3D User Interfaces* (2018). 6

[Ste92] STEUER J.: Defining virtual reality: dimensions determining telepresence. URL: https://api.semanticscholar.org/CorpusID:58000326. 6

[SW97] SLATER M., WILBUR S.: A framework for immersive virtual environments (five): Speculations on the role of presence in virtual environments. *Presence: Teleoperators & Virtual Environments 6* (1997), 603–616. URL: https://api.semanticscholar.org/CorpusID:9437981. 6

[SW20] SMITH G., WHITEHEAD J.: Game development education: Challenges and opportunities. *Communications of the ACM 63*, 6 (2020), 28–31. 2

[Swe92] SWEDISH GOVERNMENT: Higher education act (högskolelagen). SFS 1992:1434, 1992. With later amendments. 2

[Swe93] SWEDISH GOVERNMENT: Higher education ordinance (högskoleförordningen). SFS 1993:100, 1993. With later amendments. 2

[Tho11] THORN A.: *Game engine design and implementation*. Jones & Bartlett Publishers, 2011. 1

[Uni26a] UNITY TECHNOLOGIES: Unity Game Engine. https://unity.com, 2026. Accessed 2026. 5

[Uni26b] UNIVERSITY OF GOTHENBURG: Game engine architecture. https://www.gu.se, 2026. Course description and syllabus, accessed 2026. 2

[Uni26c] UNIVERSITY OF HELSINKI: Game engine architecture. https://www.helsinki.fi, 2026. Course listing, accessed 2026. 2

[Uni26d] UNIVERSITY OF MICHIGAN: Eecs 498: Game engine architecture. https://eecs498.com, 2026. Public course website, accessed 2026. 2

[Uni26e] UNIVERSITY OF NEVADA, RENO: Cs381: Game engine architecture. https://www.unr.edu, 2026. Course syllabus PDF, accessed 2026. 2

© 2026 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.